Intelligent Industrial IoT Monitoring Platform

Engineering portfolio report

Author: Amin Zoroufi
Email: aminn.zoroufi@gmail.com
Configured repository: https://github.com/aminzoroufi/intelligent-industrial-iot-monitoring-platform
Version: 0.1.0
Date: 12 August 2026

Verification level: SIMULATED - no bench or field validation is claimed
License: Portfolio Source-Available; custom text remains marked for legal review

Low-voltage demonstrator only. No safety, bench, or field certification.

1. Executive summary

This portfolio system demonstrates continuous condition monitoring for a small workshop motor, pump, or fan. It combines temperature, vibration, and DC current measurements with bounded offline buffering, MQTT transport, PostgreSQL persistence, authenticated operator workflows, Modbus RTU interoperability, and an intentionally modest Isolation Forest comparison detector.

The implementation is evidence-led. Forty-seven Python tests pass after the hardware static checks are included, the dashboard passes lint, strict TypeScript, and four component tests, and a live Compose backend slice previously exercised MQTT ingestion, replay, authentication, WebSocket delivery, alarms, and safe simulated relay acknowledgements. Embedded target builds, current CAD ERC/DRC, post-change Compose execution, physical bench behavior, and field performance remain explicitly unverified.

2. Maintenance problem and product rationale

Small workshops often inspect motors and fans manually. Bearing degradation, overload, blocked cooling, loose connections, or a failed sensor can develop between rounds. A full SCADA or predictive-maintenance suite can be disproportionate for one low-value asset, while unreliable connectivity makes a cloud-only design fragile.

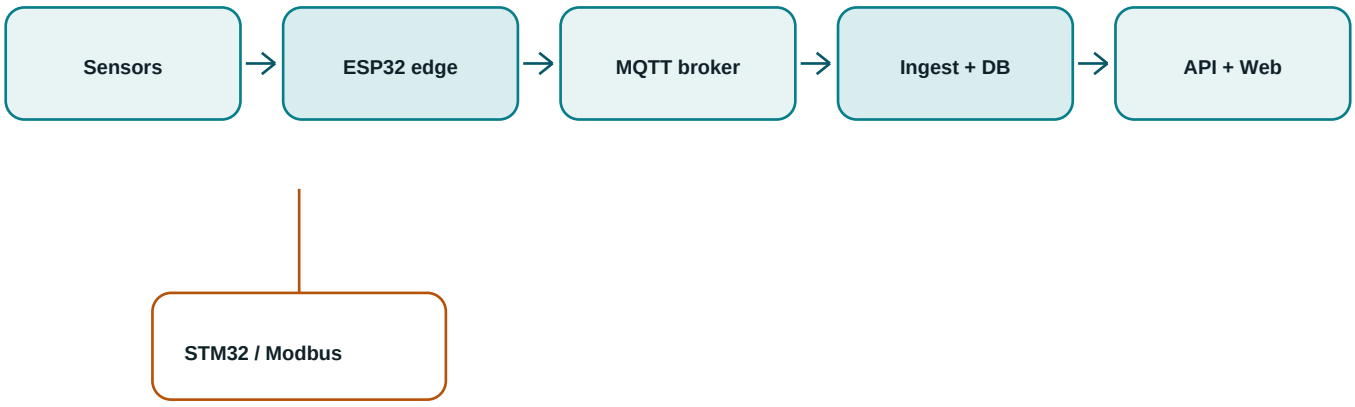
The project was chosen because it crosses embedded timing, industrial protocols, backend integrity, operator experience, electronics, and honest data analysis without requiring invented customer claims. The reference asset is the clearly simulated motor-01 at workshop-demo. The intended value is earlier, better documented maintenance attention, not automatic diagnosis or safety shutdown.

3. Stakeholders, scope, safety, and success

Stakeholders are the maintenance operator, calibration technician, administrator, and engineer evaluating retrofit feasibility. Use cases include live status, alarm acknowledgement, calibration history, maintenance records, bounded export, offline replay, and comparison of deterministic and statistical detection.

- In scope: one ESP32 gateway, one focused STM32 Modbus node, sensors, a low-voltage demo relay, local Compose services, simulation, and editable engineering sources.
- Out of scope: mains switching, emergency-stop functions, certified protection, autonomous hazardous control, customer deployment, high availability, and field-validated predictive claims.
- Success means reproducible data contracts, idempotent replay, authenticated and audited mutations, explicit health/fault state, host-tested logic, and visible verification gaps.

4. Architecture and end-to-end data flow



Explicit boundaries: SPI / I2C / UART - MQTT - SQL - REST / WebSocket

Figure 1. End-to-end architecture generated from the implemented repository boundaries.

Sensors feed the ESP32 through I2C, SPI with a data-ready interrupt, current measurement, and a UART-controlled MAX3485-class RS-485 interface. The gateway validates, aggregates, sequences, and queues telemetry before publishing versioned MQTT envelopes. A dedicated ingestor validates topic identity and JSON Schema before one transaction persists device state, telemetry, and threshold alarms.

FastAPI exposes authenticated REST and origin-checked WebSockets. PostgreSQL LISTEN/NOTIFY crosses process boundaries without turning the API into the ingestion path. The Next.js dashboard uses a same-origin backend-for-frontend for REST and an HttpOnly cookie for browser WebSocket authentication.

5. ESP32 gateway, sensors, and real-time behavior

The ESP-IDF 6.0.2 source separates production drivers from shared C logic. TMP117 and INA219 use I2C; ADXL345 uses SPI and data-ready signaling; RS-485 uses explicit driver-enable timing; a deterministic Wokwi adapter replaces only the driver boundary. FreeRTOS tasks cover acquisition, aggregation, MQTT, commands, health, and fault supervision.

Shared host logic implements configuration magic/schema/generation/CRC, migration and safe defaults, RMS/peak/crest calculations, sample-quality accounting, threshold hysteresis, bounded oldest-drop buffering, topics, serialization, and exponential reconnect with jitter. NVS uses per-slot records plus a pending journal and reserved sequence ranges to survive interruption without silent ordering ambiguity.

The relay defaults OFF, validates device and expiry, auto-clears after a bounded interval, and reports acknowledgement state. Watchdog and reboot-loop guards report reset reasons rather than hiding repeated failure. Target compilation is not claimed because ESP-IDF and a usable host C toolchain were unavailable.

6. STM32 node and Modbus RTU

The STM32F103C8T6-class node uses STM32CubeF1 HAL design intent: TIM3 triggers a 10 Hz acquisition fixture, ADC values update a versioned 18-register map, USART2 runs 115200 8E1, and RS-485 DE is explicit. Function 0x03 reads registers; bounded 0x06 writes update permitted configuration and persist a CRC-protected record in the final flash page.

Illegal function, address, value, CRC, timeout, and stale-data cases have explicit outcomes. A watchdog, backup-domain reset count, and fault flags keep operational state inspectable. Five Python tests verify golden bytes, request/response CRC, exception frames, stale registers, timeout, and a pseudo-terminal round trip. The shared C golden tests and ARM target remain build-blocked.

7. Provisioning, connectivity, queue, and health

Provisioning is intentionally local: serial commands accept bounded configuration, secrets are never returned, and factory reset requires a deliberate multi-step sequence. Wi-Fi and MQTT reconnect use bounded exponential backoff with jitter. MQTT topics include site, device, and message kind; telemetry uses QoS 1, retained health reports liveness, and LWT marks unexpected disconnects.

When the broker is unavailable, a finite flash queue records exact capacity, depth, dropped-message count, and replay flag. The oldest entry is dropped only when capacity is exhausted, and replay preserves stored order.
Clock-unsynchronized samples carry null device time and uptime rather than fabricated UTC. Heartbeats expose RSSI, reset reason/count, firmware, queue state, Modbus state, and active faults.

8. Backend data, APIs, authentication, and audit

PostgreSQL stores users, devices, thresholds, telemetry, health snapshots, alarms, calibration, maintenance, commands, audit events, error logs, and anomaly-model registry metadata. Device message ID and device-sequence uniqueness make duplicate delivery idempotent while rejecting conflicting reuse. Monotonic sequence and uptime fields are 64-bit.

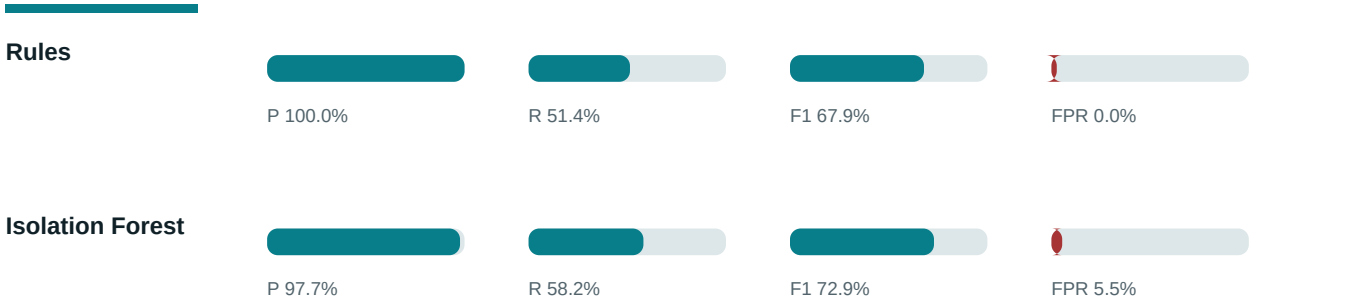
FastAPI validates bounded time ranges and payloads, returns explicit errors, escapes CSV formula prefixes, and never leaks artifact paths or secrets. OAuth2 password login issues short-lived audience-bound JWTs. Admin-only mutations create audit events. Browser WebSockets require an exact configured Origin and an HttpOnly cookie; command-line clients use a bearer subprotocol rather than query-string tokens.

9. Dashboard and maintenance workflows

The responsive Next.js 16 dashboard provides fleet status, device health, bounded/gapped telemetry charts with SI units and threshold overlays, alarm history and acknowledgement, calibration, maintenance, threshold configuration, CSV export, relay command state, and an About/legal screen. Initial independent requests run in Server Components; interactive controls remain small Client Components.

The detector-comparison route obtains model readiness from the database registry. It separates threshold, sensor-rule, and anomaly sources, reports not-ready/stale/error states, shows model version and training counts, and labels all evaluation metrics synthetic. The production build and browser sign-in/fleet smoke passed before a loopback live-channel fix; the required rebuilt browser rerun was denied, so that exact path remains blocked.

10. Anomaly model and honest interpretation



Synthetic fixture metrics - not probability and not field performance

Figure 2. Reproducible detector comparison from the versioned synthetic report.

Per-device feature schema version 1 uses a causal trailing window: temperature level/slope/variation, current mean/variation, vibration RMS mean, peak maximum, crest mean, and vibration/current ratio. Training accepts only explicitly selected good-quality, fault-free healthy windows, splits chronologically, fixes seed 20260811, and records schema, library version, interval, counts, policy, and SHA-256 checksum.

The persisted score is the negative of Isolation Forest score_samples, so higher means more anomalous. The percentile is an empirical rank against healthy validation scores, not probability. Reasons cite observed values and healthy 1st/99th percentiles without claiming root cause. Missing, stuck, noisy, impossible, inconsistent, or too-fast sensor values remain deterministic rules.

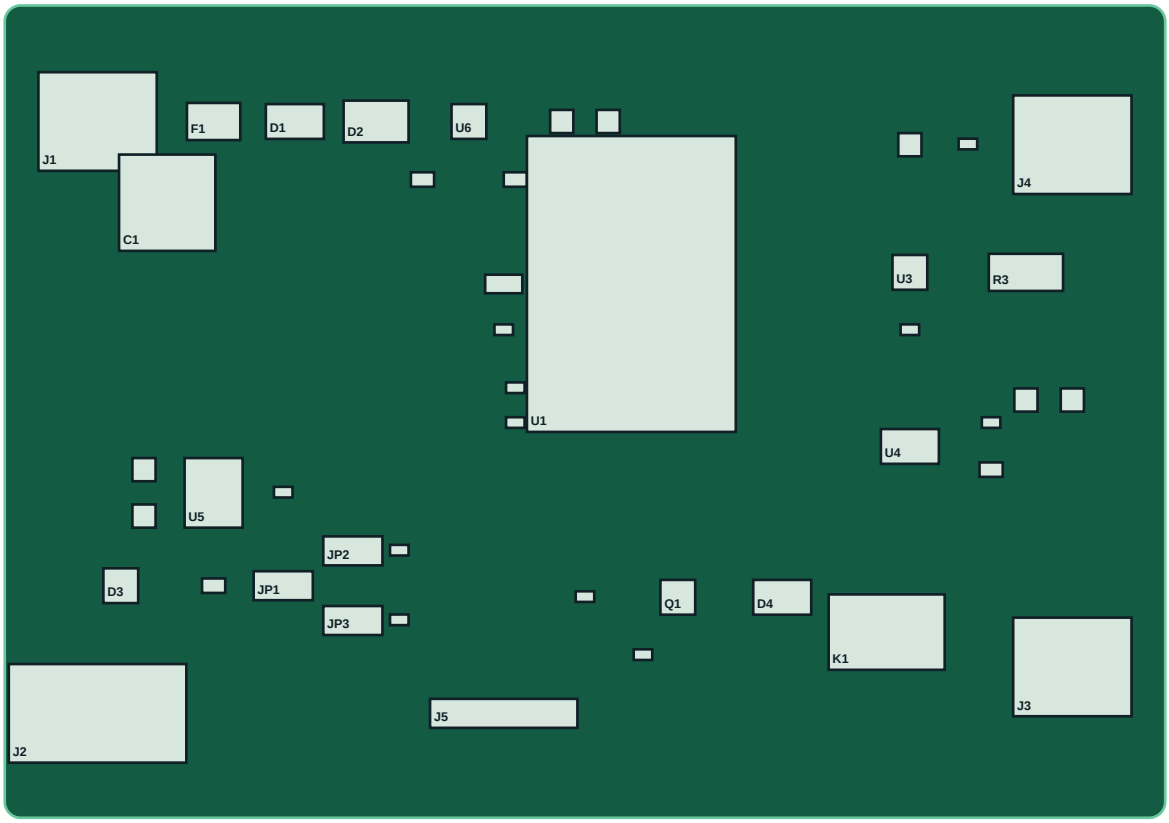
The seeded report gives model F1 0.7293 and false-positive rate 0.0545; deterministic threshold-plus-sensor F1 is 0.6787 with false-positive rate 0.0. These fixture results cannot be generalized to real equipment.

11. Simulation and exact reproduction

The telemetry generator has deterministic normal, rising-temperature, vibration-imbalance, current-overload, and stuck-sensor scenarios. Wokwi projects bind the ESP32 and STM32 production boundaries where supported; the Modbus PTY simulator provides host protocol execution without an electrical bus. Simulator gaps include RF, analog accuracy, interrupt jitter, flash wear, brownout physics, and RS-485 signal integrity.

- Copy .env.example to .env, then run docker compose up --build -d.
- Publish a named scenario with python -m simulator.telemetry_generator.main normal.
- Run Python gates with pytest, Ruff, and strict MyPy; run dashboard lint, typecheck, tests, and build separately.
- Regenerate detector evidence with python -m services.anomaly_worker.main evaluate-demo.

12. Schematic, PCB, BOM, models, and enclosure



A0-UNVERIFIED - UNROUTED - NO MAINS

Figure 3. Source-derived component placement; this is not a fabrication release.

Hardware revision A0-UNVERIFIED is a review package, not a fabrication release. The canonical design selects ESP32-WROOM-32E, TMP117, INA219 with a 50 mOhm 1 W shunt, ADXL345, MAX3485 with optional 120 ohm termination and 680 ohm bias, SM712 bus protection, AP2112K regulation, and an AO3400A/SS14 relay driver. BOM entries include MPN, rating, package, role, and sourcing warning.

The KiCad board is an un-routed review placement. The legacy schematic must be imported and saved by KiCad 10; custom pins and footprints require official-datasheet review. ERC/DRC say NOT RUN. STEP shapes named envelope are project-authored dimensional clearance shapes, not vendor models. The FreeCAD macro and generated STEP/STL enclosure share a 100 x 70 mm board envelope, but FreeCAD execution and component-accurate board export are not claimed.

13. Testing strategy and retained evidence

Area	Evidence	Status
Python	47 tests	PASS
Typed source	49 files	PASS
Dashboard	lint/type/4 tests	PASS
Live backend	earlier Compose slice	PASS
Changed Compose	follow-up not run	BLOCKED
Embedded targets	toolchains absent	BLOCKED
CAD ERC/DRC	not run	NOT RUN
Bench / field	no evidence	NOT RUN

Table 1. Verification is scoped to retained evidence; hardware skips are not passes.

Testing concentrates on integrity and failure boundaries: contract validation, timestamp consistency, duplicate/collision behavior, alarm hysteresis, authentication, authorization, WebSocket credentials, commands, Modbus bytes, feature leakage, model reproducibility, artifact tamper detection, score direction, fallback, and hardware provenance/release guards.

The host currently passes 47 Python tests with 76.31 percent branch-aware coverage against a 70 percent gate, strict MyPy over 49 product source files, Ruff, hardware static generation checks, dashboard ESLint/TypeScript, and four component tests. Earlier live Compose evidence covers PostgreSQL 18.3, Mosquitto 2.1.2, API, ingestor, simulated gateway, replay, WebSocket, alarms, and relay acknowledgements. A follow-up Compose image and browser run was not executed; no pass is inferred.

14. CI, traceability, and quality gates

GitHub Actions workflows define Python lint/type/tests/coverage, dashboard lint/type/tests/build/audit, a clean Compose/API/browser path, report generation/verification, checksum-pinned secret scanning, license policy, and hardware static checks without private credentials or hardware. Third-party actions use full commit SHAs. Hosted jobs are not called passing until the repository is published and they execute. Embedded targets and native CAD remain separate unresolved gates.

Requirements traceability maps each requirement to source, evidence, documentation, and status. The verification report distinguishes PASS, PARTIAL, BLOCKED, NOT RUN, SIMULATED, BENCH-VERIFIED, and FIELD-VALIDATED. A clean CI badge must never be used to imply a physical result.

15. Security, privacy, and electrical safety

Local demo credentials are public development values and must be rotated for any other environment. Production needs TLS/WSS, per-device broker credentials or certificates, ACLs, secret management, restricted CORS, backup/retention policy, signed update controls, and a deployment threat review. Telemetry is operational rather than personal, but operators and maintenance notes still require least privilege and retention decisions.

Joblib is a pickle-compatible trusted-artifact format. Only administrator-controlled model roots are acceptable; checksum proves integrity, not authorship. The relay must never switch mains or replace an emergency stop, interlock, personnel protection, or physical isolation. Revision A0-UNVERIFIED must not be fabricated.

16. Trade-offs, lessons, limitations, and next steps

The architecture favors explicit contracts and separate processes over a compact monolith; this adds deployment pieces but keeps ingestion, operator API, and model failure independent. Per-device models are less convenient than one global model but make baselines and fallback visible. Bounded queues choose measurable loss over uncontrolled flash growth. An inexpensive two-layer carrier is approachable but demands careful layout and validation.

Credible next steps are to accept/install toolchains, compile both targets with warnings enabled, migrate and route the KiCad design, run ERC/DRC, rebuild FreeCAD outputs, execute all Wokwi scenarios, rerun the changed Compose/dashboard path, assemble a current-limited prototype, retain calibrated bench evidence, and define a controlled field protocol before discussing predictive value.

17. Repository map, glossary, references, and appendices

Repository paths are contracts for schemas, services for backend and anomaly processes, web for the dashboard, firmware for ESP32/STM32/shared C, simulator for scenarios and Modbus/Wokwi, hardware for editable review sources, data/demo for versioned synthetic evidence, and docs for ADRs, operations, safety, traceability, and reports.

Glossary: LWT is MQTT Last Will and Testament; QoS is delivery quality of service; RMS is root mean square; CRC is cyclic redundancy check; RTU is Modbus binary serial framing; DE is RS-485 driver enable; ERC/DRC are electrical/design rule checks; SELV/PELV are extra-low-voltage safety classifications.

Primary references and exact verified tool versions are recorded in docs/toolchain-versions.md. Design decisions are in docs/adr. Reproduction details, raw-status boundaries, and command evidence are in docs/verification-report.md, docs/local-development.md, docs/anomaly-detection.md, hardware/README.md, and docs/hardware-in-the-loop.md. The configured GitHub destination is not presented as published until authentication and remote verification succeed.